



Komunikasi Serial Peripheral Interface (SPI)

Protokol komunikasi data sinkron berkecepatan tinggi dalam sistem *embedded* dari mikrokontroler ke periferal, antar prosesor, hingga sistem penyimpanan eksternal.

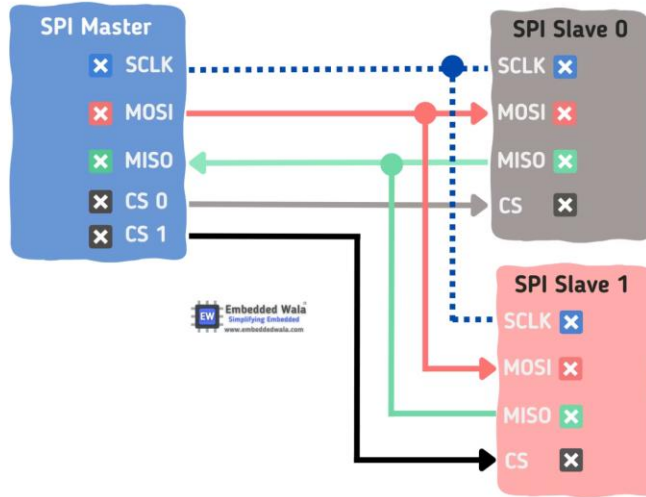
FAHREZA ILHAM TANGGUH ADITYA

40040324620007

PRAKTIKUM A

TRO 24

Apa Itu SPI?



Serial Peripheral Interface (SPI) adalah protokol komunikasi data sinkron berkecepatan tinggi yang beroperasi secara **dupleks penuh**, dapat mengirim dan menerima data secara bersamaan. Dalam perancangan sistem kendali dan *embedded systems*, antarmuka ini sangat krusial karena kemampuannya mentransfer data secara cepat antara satu perangkat utama (Master) dan satu atau beberapa perangkat pendukung (Slave). Fleksibilitas SPI menjadikannya **standar industri** untuk menghubungkan mikrokontroler dengan berbagai periferal, mulai dari layar antarmuka, modul memori eksternal, hingga komunikasi antar mikrokontroler.

Kecepatan Tinggi

Transfer data sinkron dengan clock yang dibangkitkan master

Dupleks Penuh

Mengirim dan menerima data secara bersamaan

Multi-Slave

Satu master mengendalikan beberapa perangkat slave

Empat Jalur Sinyal Utama SPI

Arsitektur SPI menggunakan empat jalur sinyal utama yang bekerja bersama untuk mentransfer data antara master dan slave.



MOSI

Master Out Slave In - Jalur pengiriman data dari master ke slave



MISO

Master In Slave Out - Jalur penerimaan data dari slave ke master



SCK

Serial Clock - Detak sinkronisasi yang dibangkitkan oleh master



CS / SS

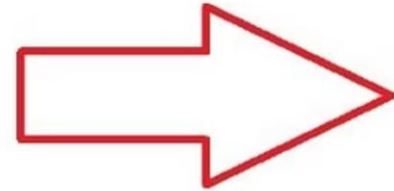
Chip Select / Slave Select - Mengaktifkan perangkat slave tertentu di dalam bus komunikasi

nect TFT 2.8 inch L

PERCOBAAN 1

Koneksi ESP32-WROOM ke TFT LCD

Pada percobaan pertama, antarmuka SPI digunakan untuk mengendalikan tampilan grafis pada layar TFT LCD menggunakan ESP32-WROOM sebagai master. Layar TFT umumnya bertindak sebagai perangkat **Write-Only** dalam pengoperasian dasar, sehingga sinyal komunikasi lebih banyak mengalir dari mikrokontroler menuju layar melalui jalur MOSI dan disinkronkan oleh SCK.



Alur Program: ESP32 → TFT LCD



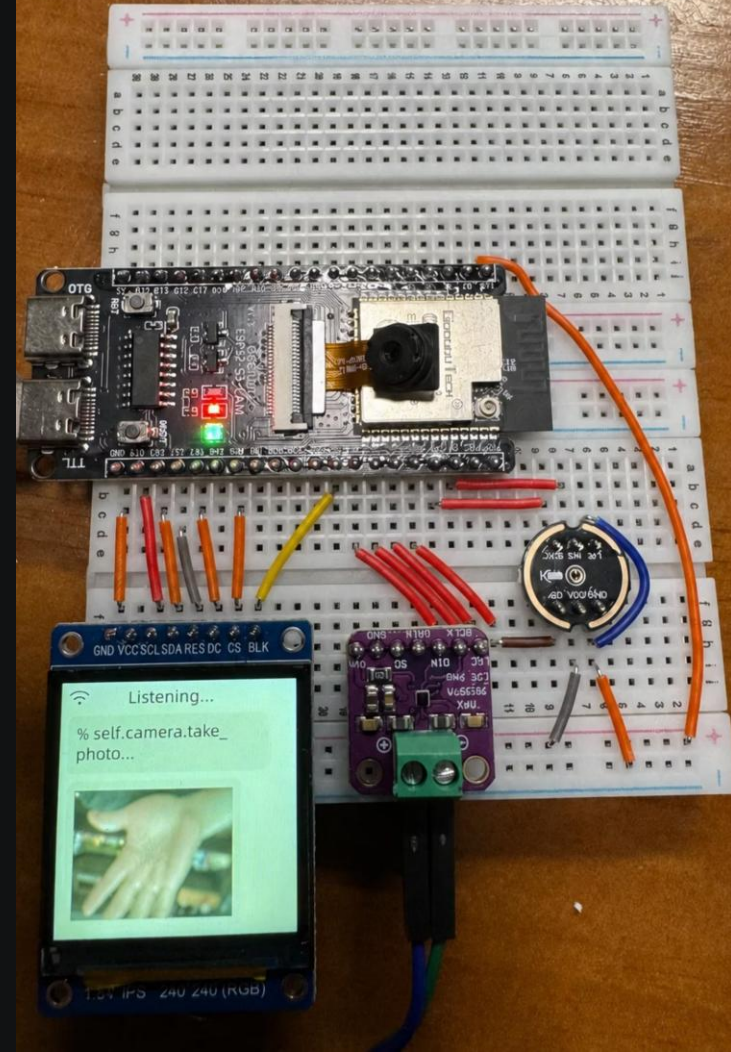
Pada tingkat perangkat lunak, program diawali dengan menginisialisasi pin-pin yang akan digunakan sebagai jalur SPI alternatif (misalnya menggunakan bus VSPI atau HSPI pada ESP32). Program dikonfigurasi melalui pemanggilan fungsi inisialisasi layar yang mengatur pin Chip Select (CS) pada kondisi logika tinggi (HIGH) sebagai keadaan siaga.

Ketika program ingin menggambar piksel atau mengirimkan warna, mikrokontroler akan menarik pin CS menjadi logika rendah (LOW) untuk mengaktifkan layar. Selanjutnya, program mengirimkan sebuah *byte* perintah (misalnya perintah untuk mengatur jendela kolom dan baris pada memori layar) yang diikuti oleh aliran *byte* data yang mewakili nilai warna RGB. Sinyal tambahan yaitu pin **Data/Command (DC)** dikendalikan secara bersamaan di dalam program untuk memberi tahu layar apakah *byte* yang dikirimkan via jalur MOSI merupakan sebuah instruksi pengaktifan atau data warna murni. Setelah transfer selesai, program akan mengembalikan pin CS ke logika tinggi agar bus SPI dapat dibebaskan.

PERCOBAAN 2

Komunikasi Antar Mikrokontroler: ESP32 → STM32

Percobaan kedua mendemonstrasikan pertukaran data secara langsung antara dua otak pemrosesan yang berbeda arsitektur, **ESP32 sebagai Master** dan **STM32 (F407VGT6/F103) sebagai Slave**, dengan tujuan mengirimkan string pesan *"Hello World"* secara utuh dan terstruktur.



Alur Program: ESP32 (Master) ↔ STM32 (Slave)

⚡ ESP32 - Sisi Pengirim (Master)

Di dalam penyusunan logika programnya, algoritma dibagi menjadi dua bagian. Pada mikrokontroler ESP32 (Master), program mendefinisikan array karakter yang berisi kalimat "Hello World". Dalam putaran utama (*main loop*), ESP32 menarik pin CS Slave (STM32) menjadi LOW, lalu menggunakan perulangan *for* untuk mengirimkan karakter satu per satu. Setiap karakter dimasukkan ke dalam register pengiriman SPI secara berurutan, selaras dengan detak clock yang dihasilkan ESP32.

🔔 STM32 - Sisi Penerima (Slave)

Di sisi penerima (STM32), program dikonfigurasi dalam mode SPI Slave dengan mengaktifkan fitur **Interrupt** (Interupsi). Program STM32 tidak terus-menerus mengecek status secara manual (*polling*), melainkan akan memicu sebuah fungsi penanganan interupsi (*interrupt handler*) setiap kali ada *byte* masuk ke dalam register MISO/MOSI. Di dalam fungsi interupsi tersebut, STM32 akan membaca karakter yang datang dan merangkainya kembali ke dalam sebuah *buffer* memori lokal hingga karakter penutup terdeteksi, yang kemudian string "Hello World" tersebut dapat dicetak melalui serial monitor untuk proses *debugging*.



Master Kirim

Bit per Bit via MOSI

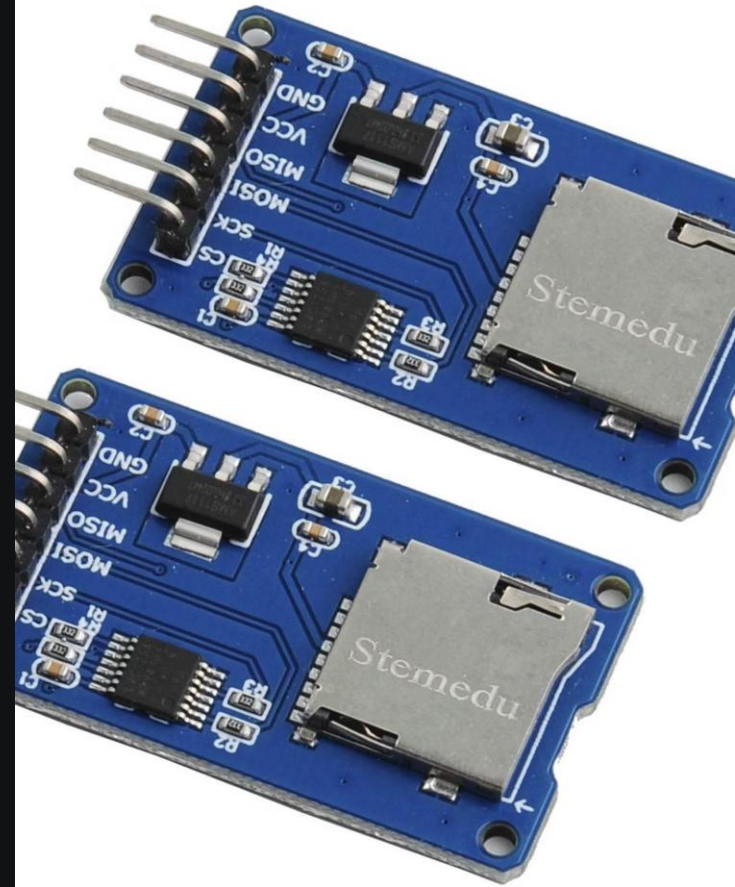
Clock SCK

Slave Terima & Buffer

PERCOBAAN 3

Koneksi untuk Membaca Modul SD Card Reader

Percobaan ketiga memanfaatkan protokol SPI untuk mengakses memori eksternal (SD Card) menggunakan mikrokontroler. Berbeda dengan layar TFT yang didominasi oleh penulisan data, akses SD Card sangat bergantung pada jalur **MISO** karena mikrokontroler harus membaca balasan perintah, status sektor memori, hingga isi file secara masif.



Alur Program: Membaca SD Card via SPI

Mekanisme program untuk membaca SD Card jauh lebih kompleks secara logis. Berikut adalah tahapan lengkapnya:

01

Inisialisasi Clock Rendah (~400 kHz)

Program mengawali komunikasi dengan mode inisialisasi di mana frekuensi clock (SCK) diturunkan secara drastis (biasanya di kisaran 400 kHz) untuk kompatibilitas awal pembacaan kartu.

03

Naikkan Clock ke Kecepatan Maksimal

Setelah kartu SD merespons dan berhasil terinisialisasi, program mengubah pengaturan clock SPI menjadi kecepatan maksimal.

02

Kirim Dummy Bytes (0xFF)

Program kemudian mengirimkan serangkaian *dummy bytes* (biasanya bernilai logika 1 semua atau 0xFF) pada jalur MOSI secara terus menerus agar SD Card bisa membalas dengan status kesiapannya.

04

Baca File via MISO

Untuk membaca sebuah file teks, program akan mengirimkan instruksi pembacaan blok data. Selama proses pembacaan file berlangsung, mikrokontroler terus mengirimkan *dummy bytes* di jalur MOSI hanya untuk menjaga siklus clock SCK tetap berjalan, sementara program secara aktif menyimpan aliran *byte* yang masuk dari jalur MISO ke dalam variabel penampung (*array buffer*) hingga parameter penanda akhir file (*End of File*) diterima.

Integrasi TFT LCD + SD Card dengan FreeRTOS

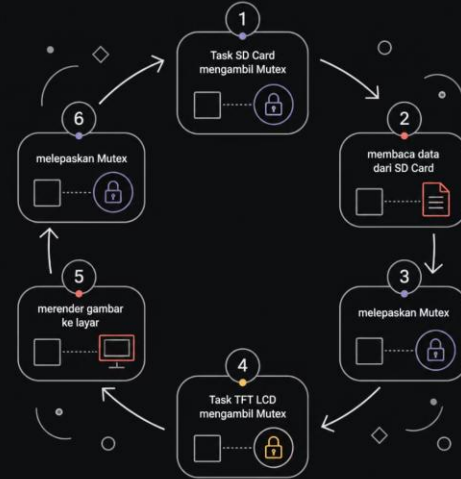
Manajemen Bus SPI Bersama dengan Mutex

Pada percobaan keempat, satu bus komunikasi SPI (MOSI, MISO, SCK) digunakan secara bersamaan oleh lebih dari satu perangkat (*multiple slaves*), yaitu layar TFT LCD dan SD Card reader, dengan manajemen waktu yang diatur menggunakan sistem operasi waktu nyata (FreeRTOS) pada ESP32.

Konsep utama dari program ini adalah sinkronisasi dan pembagian sumber daya agar data gambar tidak bertabrakan dengan data file. Program tidak menggunakan fungsi *delay* konvensional, melainkan mendeklarasikan dua buah *Task* secara paralel:

- **Task1** - Memuat data gambar atau konfigurasi dari dalam SD Card
- **Task2** - Me-render visual ke layar TFT LCD

Untuk mengamankan bus SPI yang digunakan bersama, program memanfaatkan objek sinkronisasi bernama **Mutex** (*Mutual Exclusion*). Ketika Task SD Card ingin membaca data, program akan memerintahkan Task tersebut untuk "mengambil" Mutex. Selama Mutex ini ditahan, Task layar TFT akan otomatis berstatus menunggu (Block/Suspend) dan pin CS layar TFT dipastikan berada pada posisi logika tinggi. Setelah SD Card selesai mentransfer file dan CS SD Card dinonaktifkan, program akan melepaskan Mutex tersebut. Saat itulah, Task TFT LCD langsung "mengambil" Mutex, menarik pin CS layar menjadi LOW, dan menggunakan jalur SPI untuk menggambar antarmuka, sementara SD Card menunggu gilirannya. Siklus pergantian status penguncian jalur ini terjadi dalam hitungan milidetik.



Kesimpulan

Protokol SPI terbukti sebagai metode pertukaran data yang sangat *versatile* dan andal dalam ekosistem sistem tertanam (*embedded system*). Melalui serangkaian percobaan di atas, dapat disimpulkan bahwa SPI mampu menangani berbagai beban kerja yang berbeda, mulai dari transfer data searah yang cepat untuk aktuasi visual (TFT LCD), komunikasi sinkron dua arah yang presisi antar unit pemrosesan utama (ESP32 dan STM32), hingga pengambilan data bervolume besar pada sistem penyimpanan eksternal (SD Card). Ketika sistem semakin kompleks dan menuntut beberapa perangkat periferal beroperasi pada jalur perangkat keras yang sama, perpaduan SPI dengan penjadwalan **Real-Time Operating System** (RTOS) melalui sistem *locking* seperti Mutex memastikan integritas data tetap terjaga tanpa mengurangi efisiensi dan kecepatan sistem secara keseluruhan.